

Assignment: The Great Land Partition

Python Programming - Week 9

1. The Story

The King has n blocks of stone of different sizes that must be joined into one massive monument. These blocks are arranged in a specific sequence.

Each block i is defined by its dimensions $(p_{i-1} \times p_i)$. When you join two adjacent blocks (or already joined groups of blocks), the cost is determined by the outer dimensions and the shared boundary. Specifically, joining a group with dimensions $(p_{i-1} \times p_k)$ and another with $(p_k \times p_j)$ costs $p_{i-1} \cdot p_k \cdot p_j$. Your goal is to find the sequence of joins that results in the **minimum total cost**.

2. Problem Statement

Given a sequence of dimensions $P = [p_0, p_1, \dots, p_n]$ for n blocks:

1. Calculate the minimum total cost to join all blocks.
2. Output the index k of the very last join made to achieve this minimum cost.

3. Technical Constraints

- $2 \leq n \leq 100$ (Number of parcels).
- $1 \leq p_i \leq 500$ (Dimension values).
- Time Complexity: $O(n^3)$.
- Space Complexity: $O(n^2)$ for the DP table and split table.

4. Input & Output Format

Input:

- A single line of $n + 1$ space-separated integers representing the dimensions P .

Output:

- Line 1: The minimum cost (an integer).
- Line 2: The optimal split for the full range (i.e., the value of k that split the very first calculation of $[1, n]$).

Example:

Input:

40 20 30 10 30

Output:

26000

3

(Explanation: The optimal way to join these 4 blocks involves a final split at the 3rd index.)

5. Implementation Skeleton

Submit a file named `week9_[yourRollnumber].py`.

```
1 """
2 Name: [Your Name]
3 Roll Number: [Your Roll Number]
4 Time Complexity Analysis:
5 Explain why this Interval DP approach is  $O(n^3)$ .
6 """
7
8 def solve_land_partition(p):
9     n = len(p) - 1
10    # m[i][j] stores the minimum cost
11    m = [[0] * (n + 1) for _ in range(n + 1)]
12    # s[i][j] stores the index of the optimal split k
13    s = [[0] * (n + 1) for _ in range(n + 1)]
14
15    # TODO: Implement Interval DP logic
16    # 1. Loop for chain length 'L' from 2 to n
17    # 2. For each length, find the minimum cost for range [i, j]
18    # 3. Store the best 'k' in s[i][j]
19
20    return m[1][n], s[1][n]
21
22 if __name__ == "__main__":
23     dims = list(map(int, input().split()))
24     min_cost, best_k = solve_land_partition(dims)
25     print(min_cost)
26     print(best_k)
```

6. Submission Instructions

- **File Name:** `week9_[yourRollnumber].py`
- **Requirement:** You must provide a clear Big O analysis in the comment block. Explain how the nested loops contribute to the $O(n^3)$ complexity.